CODETANTRA

AN INTERNSHIP REPORT ON

"Machine Learning Using Python Internship"

Submitted in partial fulfillment of requirements to AIDS

II/IV B. Tech AIDS

Submitted by

P. DIVYA (23KQ1A54F6)

JUNE 2025

PACE INSTITUTE OF TECHNOLOGY AND SCIENCE

PACE INSTITUTE OF TECHNOLOGY AND SCIENCE

DEPARTMENT OF AIDS

This is to certify that this internship report **"Machine Learning Using python Internship"** is the Bonafide work of **"P. DIVYA (23KQ1A54F6)"** who has carried out the work under my supervision and submitted in partial fulfillment for the award of **Codetantra Machine Learning Internship** during the year 2024(May)-2024(June).

Signature of Internal Trainer:

Signature of External Trainer:

ACCIDENT DETECTION SEVERITRY PREDICTION USING MACHINE LEARNING

SEMESTER INTERNSHIP

CSE-ARTIFICIAL INTELLIGENCE & DATA SCIENCE

SUBMITTED BY: **P. DIVYA**

G. RAMYA	**24KQ5A5415**
K. SATYA SRI	23KQ1A54E2
P. JAHNAVI	23KQ1A54F5
P. DIVYA	23KQ1A54F6

UNDER GUIDANCE OF

D. HARSHINI

P A C E

INSTITUTE OF TECHNOLOGY & SCIENCES

(AUTONOMOUS)

Approved by AICTE, Accredited by NBA & NAAC (A Graded), Recognized under 2(f) &12(B) of UGC permanently Affiliated to JNTUK, Kakinada .A.P., An ISO 9001:2008 certified Institution

ACKNOWLEDGEMT

I would like to express my sincere gratitude to all those who supported and guided me throughout my internship journey tled “Machine Learning Using Python Internship.”

First and foremost, I extend my heart felt thanks to **(Dr. G.V.K. MURTHY)**, Principal of PACE Ins tute of Technology and Science, for providing me with an encouraging environment and the opportunity to undertake this internship.

I am deeply grateful to **(G. GANESH NAIDU)** Head of the AIDS, for paving the path and suppor ng me with all the necessary resources and guidance required to complete this internship.

My sincere thanks to **(K. RAJ KIRAN)** internship In-charge, for his valuable sugges ons, constant encouragement, and insigh ul feedback that played a significant role in the successful comple on of this internship.

I would like to place on record my sincere apprecia on and gra tude to Codetantra, the organiza on where I had the opportunity to pursue my internship. I am thankful for the exposure, technical learning, and real- me insights into Machine Learning that I gained during this period.

Finally, I amimmensely thankful to my friends and family for their unwavering moral support throughout this journey.

.

P. DIVYA

23KQ1A54F6

ABSTRACTION

This internship provided comprehensive exposure to fundamental and advanced concepts in Machine Learning, with a strong emphasis on prac cal implementa on and hands-on experience. The learning journey began with core data analysis libraries such as NumPy, Pandas, Seaborn, and Matplotlib, focusing on data preprocessing and data visualiza on techniques.

The curriculum covered an in-depth overview of Machine Learning, its categories, and real-world applica ons. The types of machine learning explored included Supervised Learning, Unsupervised Learning, and Reinforcement Learning.

In Supervised Learning, I learned about various algorithms such as:

- Candidate Elimina on Algorithm
- Naive Bayes Classifier and Bayes Theorem
- k-Nearest Neighbors (k-NN)
- Classifica on and Regression techniques

In Unsupervised Learning, the focus was on Clustering, including:

- Types of clustering techniques
- Implementa on of K-Means Clustering with real-life examples and code
- Dimensionality Reduc on (PCA)

Reinforcement Learning concepts were also introduced, including:

- Q-Learning and its real-world applica ons

III-

- An **accident detection and severity prediction** system uses sensor data and machine learning to identify accidents in real time.
  It gathers data from GPS, accelerometers, and vehicle telemetry to detect abnormal events.
  Upon detection, severity is predicted using trained models analyzing impact-related features.
  The system enables faster emergency response and efficient allocation of resources.

TABLE OF CONTENTS

DESCRIPTION:

The "Accident Detection and Severity Prediction Using Machine Learning" project aims to enhance road safety by leveraging advanced data analytics and machine learning (ML) techniques to predict the occurrence and severity of traffic accidents. With the increasing number of vehicles on roads and the growing incidence of traffic collisions, it has become essential to develop intelligent systems that can analize traffic data and provide timely insights for preventive measures.

In this project, real-world traffic accident datasets containing various attributes such as weather conditions, road surface types, vehicle types, lighting conditions, time of day, and traffic flow are used. These features are pre processed and fed into different machine learning algorithms to train predictive models capable of classifying accident severity levels into categories such as Slight, Serious, and Fatal.

The project follows a structured machine learning pipeline including data preprocessing, feature selection, model training, evaluation, and deployment. Various algorithms like Random Forest, Support Vector Machine (SVM), and Multi-Layer Perceptron (MLP) are tested to determine the most accurate model. The performance of the models is evaluated using metrics like accuracy, precision, recall, and F1-score.

By accurately predicting the severity of potential accidents, the system can support authorities in resource allocation, emergency response planning, and traffic control strategies. Furthermore, it can be integrated with IoT sensors or smart city infrastructure to provide real-time alerts and support accident prevention initiatives.

Road traffic accidents are one of the leading causes of injury and death globally. The increasing number of vehicles and expanding road networks have made it more challenging to monitor traffic and ensure safety effectively. In such a scenario, advanced technologies like **Machine Learning (ML)** offer promising solutions to detect accidents in real-time and predict their severity based on various influencing factors. This project aims to develop an intelligent system using ML techniques that can analyze traffic accident data to detect incidents and classify their severity into categories such as **Slight**, **Serious**, or **Fatal**.

In an increasingly interconnected world, road safety remains a critical concern, with accidents causing significant human and economic loss. Our project, "Intelligent Accident Detection and Severity Prediction System," harnesses the power of Machine Learning to revolutionize how we respond to and mitigate the impact of road incidents.

- **Proactively Detect Accidents:** Employing advanced computer vision techniques (e.g., analyzing CCTV feeds, dashcam footage) and sensor data fusion (e.g., accelerometer, GPS from vehicles/smartphones), the system can instantly identify the occurrence of a vehicular accident. This capability is crucial for overcoming the limitations of manual reporting and ensuring immediate awareness.
- **Accurately Predict Accident Severity:** Beyond mere detection, our core innovation lies in predicting the severity of an accident. Through the application of sophisticated Machine Learning models (e.g., classification algorithms like Random Forest, Gradient Boosting, or deep learning architectures), we analyze a multitude of factors including:
- **Accurately Predict Accident Severity:** Beyond mere detection, our core innovation lies in predicting the severity of an accident
- **Vehicle Dynamics:** Speed at impact, type of collision (head-on, rear-end, side-swipe), vehicle rollover.
- **Environmental Conditions:** Road surface conditions (wet, dry, icy), visibility (fog, rain), time of day.
- **Facilitate Rapid Emergency Response:** Upon detection and severity prediction, the system is engineered to automatically trigger alerts to designated emergency services (police, ambulance, fire department). This immediate notification, coupled with precise location data and predicted severity, drastically reduces response times, a critical factor in minimizing injuries and fatalities.

- **Enhance Post-Accident Analysis and Prevention**: The data collected and analized by our system provides invaluable insights for urban planners, traffic engineers, and policymakers. By identifying accident hotspots, common contributing factors, and high-risk scenarios, the system serves as a powerful tool for developing targeted prevention strategies, improving road infrastructure, and informing public safety campaigns.

Machine Learning (ML):

It is a revolutionary paradigm within Artificial Intelligence (AI) that empowers computers to **learn from experience** without being explicitly programmed for every scenario. ML relies on robust mathematical and statistical principles. Different types of ML (like supervised, unsupervised, and reinforcement learning) employ various algorithms (e.g., neural networks, decision trees, support vector machines) tailored to specific learning tasks.

Instead of meticulously coding each step for a specific task, ML allows us to build systems that can:

1. **Identify Patterns in Data:** ML algorithms are fed massive amounts of data – numbers, images, text, sounds, sensor readings, and more. This "pattern recognition" is the bedrock of what ML can achieve.
2. **Make Intelligent Predictions and Decisions:** Once a machine learning model has "learned" from the data, it gains the ability to make informed predictions or decisions on *new, unseen data*.
3. **Continuously Improve and Adapt:** A key differentiator of ML is its iterative nature. As more data becomes available, the model can be retrained and refined, leading to improved accuracy and performance over time. This continuous learning capability makes ML systems incredibly powerful and adaptable, especially in dynamic environments

4. **Automation of Complex Tasks:** From automating mundane data entry to powering sophisticated robotic systems, ML automates tasks that were once exclusively in the human domain.

5. **Unveiling Hidden Insights:** It extracts valuable knowledge from enormous datasets, enabling better decision-making in business, science, and healthcare.

6. **Solving Previously Intractable Problems:** ML provides solutions for challenges that were considered too complex for traditional programming, such as real-time language translation, autonomous driving, and advanced medical diagnostics.

7. **Predictive Power:** Its ability to forecast future events or behaviours, from stock market fluctuations to equipment failures, offers unprecedented opportunities for proactive planning and risk mitigation

ML (Work flow) for Accident detection and Severity prediction:

The development of an effective Machine Learning system for accident detection and severity prediction involves a systematic, multi-stage workflow. Each phase is crucial for building a robust, accurate, and deployable solution. Here's an in-depth look at the typical ML workflow.

1. Understand the Goal (What do we want to do?):

- **Accident Detection:** We want our system to yell, "Accident!" as soon as one happens.

. **Severity Prediction:** And then immediately say, "It's minor!" or "It's serious!" so help can be sent quickly.

2. Gather the Clues (Data Collection):

- We need lots of examples of both **normal driving** and **actual accidents**.
    - What the sensors did during the crash.
    - **How bad** the accident really was (e.g., minor fender-bender, severe injury, fatality). This is our "answer key."

3. Clean & Prepare the Clues (Data Preprocessing & Feature Engineering):

- **Clean Up:** Imagine messy notes. We need to fix typos, fill in missing parts, and get rid of anything that looks wrong or useless.
- **Label Them:** For every accident example, we *must* clearly mark if it was "minor," "moderate," or "severe."

4. Teach & Test the Brain (Model Training & Evaluation):

It studies these examples, learns the patterns, and figures out how to tell normal driving from accidents, and how to guess severity.

5. Fine-Tune the Brain (Model Optimization):

- We try to make sure it's not just memorizing our examples (overfitting) but truly understanding the patterns so it can handle *new* situations.

6. **Launch the Assistant! (Deployment):**

Now, when something happens, the system can instantly analyze the live data and send alerts to emergency services with the detected accident and its predicted severity.

7. **Keep it Smart (Monitoring & Maintenance**):

If it starts making more mistakes, we collect new data and "retrain" it, just like updating an app to make it better. This keeps it accurate over time.

DATASET USED:

RTA Dataset.csv refers to the **Road Traffic Accident dataset**, commonly used in accident analysis and prediction tasks. It typically contains records of road accidents and related features collected over time by transportation or safety authorities.

This dataset is used in **Supervised Learning**.

This is a **Classification** problem, because:

- The target variable (Accident_ severity) is **categorical** (e.g., Slight, Serious, Fatal).
- The aim is to classify new accident records into one of these categories.
- ◆ **Structure of the Dataset:**
- The dataset is presented in **CSV (Comma-Separated Values)** format and consists of several features that describe conditions under which road accidents occurred. Some of the most important columns include.

Purpose of the Dataset:

- Analyze causes of traffic accidents.
- Predict the severity of accidents using Machine Learning models.
- Identify the most common accident scenarios.
- Improve road safety by understanding contributing factors.

Benefits of Using the RTA Dataset:

- **Real-world application**: Helps simulate real scenarios of traffic management.
- **Ideal for ML models**: Can be used with algorithms like Random Forest, SVM, Neural Networks, etc.
- **Supports accident prevention**: Insights derived can be used by traffic authorities for preventive measures.

Common Use Cases:

- Building an accident severity prediction system.
- Developing chatbots for accident severity prediction (like your Telegram bot).

Source of the Dataset: Available on Kaggle, which provides a real-world dataset collected by the Ethiopian government from 2006–2016.

ALGORITHM:

1. **Random Forest Classifier:**
   - It is supervised learning
   - It is a classification problem

• **Library**: sklearn.ensemble.Random Forest Classifier

• **Description**: An ensemble learning method that builds multiple decision trees and merges them to get a more accurate and stable prediction.

• **Why it's used**:

- Works well with both categorical and numerical features.
- Handles missing values well.
- Good Accuracy for Classification problem.

2. **Support Vector Machine (SVM) :**

- It is supervised learning
- It is a classification problem

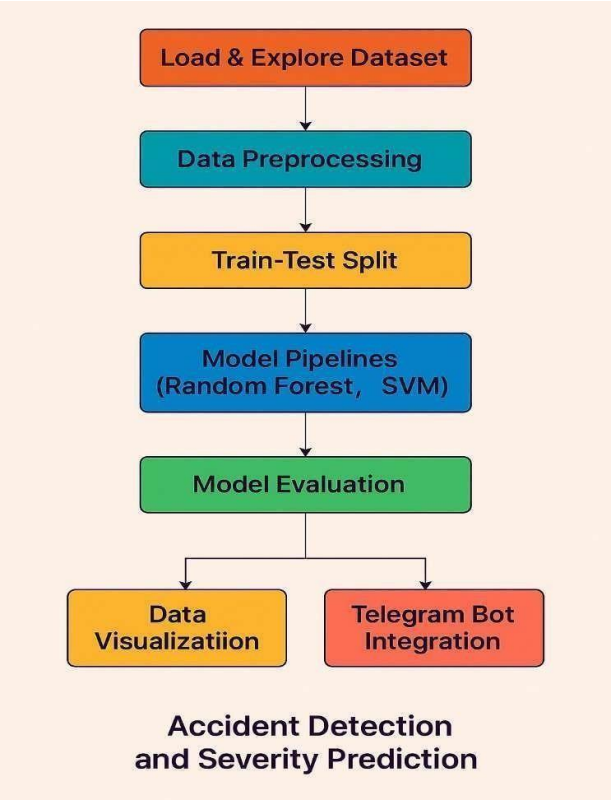
• **Library**: sklearn.svm.SVC

• **Kernel**: "rbf" (Radial Basis Function)

• **Description**: SVM is a powerful classifier that tries to find a hyperplane that best separates different classes.

• **Why it's used**:

- Performs well in high-dimensional spaces.
- Effective in non-linear classification using kernel tricks like RBF.
- Robust against overfitting in lower-dimensional space.



3. Neural Network (Multi-Layer Perceptron - MLP):

- **Library: sklearn.neural_network.MLP Classifier:**
  - **Description**: A type of feedforward artificial neural network that uses backpropagation to learn from data.
  - **Why it's used**:
    - Can model complex non-linear relationships.
    - Learns from patterns in the data.
    - Suitable for classification problems with multiple classes.

Real-Time Example:

- **Accident Severity Prediction via Telegram Bot for Emergency Dispatch**

Scenario:

A **traffic control center** or **emergency response team** uses a **Telegram bot** to **quickly assess accident severity** using real-time accident data input from field officers or automated sensors.

EXISTING SYSTEM:

1. **Manual Reporting:**
- Accidents are reported by individuals via phone calls or messages.
- Delays in response time can occur due to human factors.

2. **Basic Surveillance Systems:**
- Use of CCTV footage or traffic police observation to detect accidents.
- Requires manual monitoring and analysis.
- Lack of real-time severity assessment.

3. **No Predictive Capability:**
- Existing systems often don't predict the **severity** of accidents.
- No use of machine learning to forecast or classify severity levels.

4. **Limited Automation:**
- Decision-making is slow and reactive.
- Minimal integration with mobile or cloud-based technologies.

PROPOSED SYSTEM:

1. **Automated Detection & Prediction:**
    a. Machine Learning models (Random Forest, SVM, Neural Network) trained on accident datasets.
    b. Automatically predicts accident **severity levels** based on features like:
        i. Weather conditions
        ii. Road surface
        iii. Type of vehicle iv. Light conditions
2. **Integrated Communication (Telegram Bot)**:
    a. Users can interact with the system through a Telegram bot.
    b. Results of model predictions are directly sent to the user.
    c. Helps in faster communication of critical data.
3. **Real-Time Data Processing**:
    a. Preprocessed data is classified quickly using ML models.
    b. Can be extended to work with real-time streaming data from sensors or IoT devices.
4. **Scalability & Flexibility**:
    a. Easily extendable to new regions or datasets.
    b. Can be upgraded with deep learning or computer vision modules in the future.
5. **Improved Decision Making**:
    a. Accurate predictions help emergency services prioritize severe accidents.
    b. Useful insights can be derived for city planning and road safety improvements.

ADVANTAGES:

1. Fast and Real-Time Detection
2. Accurate Severity Prediction
3. Data-Driven Decision Making
4. Automation Reduces Human Error
5. Easy User Interaction (via Telegram Bot)
6. Scalable and Adaptable
7. Continuous Learning and Improvement
8. Cost-Effective in the Long Term
9. Easy Integration with IoT Devices

DIS-ADVANTAGES:

1. Model Accuracy Depends on Data Quality
2. Black-Box Nature of ML Models
3. High Computational Resources
4. Dependency on Internet/Data Access
5. Needs Frequent Updating
6. Privacy and Security Concerns
7. Complex Setup and Maintenance
8. Limited by Dataset Scope

Architecture for Accident Detection and Severity Prediction

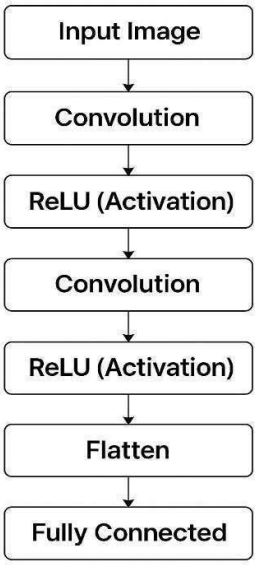
The architecture for accident detection and severity prediction is designed to handle data collection, preprocessing, machine learning model training, evaluation, and real-time interaction. It is composed of the following key components:

1. **Data Collection Layer**
   - The process begins with collecting accident-related data from reliable sources such as traffic departments, open datasets, or IoT devices.
   - Typical data fields include **weather conditions, road surface status, vehicle type, light conditions**, and **accident severity**.
2. **Data Preprocessing Layer**
   - Raw data often contains missing values and inconsistencies.
   - This layer handles:
     - **Missing value imputation**
     - **Encoding categorical variables**
     - **Feature scaling**
   - A Column Transformer is used to combine numerical scaling and categorical encoding in one pipeline.
3. **Model Training & Evaluation Layer**
   - The processed dataset is split into training and test subsets.
   - Several machine learning models such as:
     - **Random Forest Classifier**
     - **Support Vector Machine (SVM)**
     - **Neural Networks**
       are trained on the dataset.
   - Models are evaluated using **accuracy scores** and **classification reports** to select the best-performing algorithm.
4. **Prediction & Visualization Layer**

- Once trained, the model is used to predict the severity of new accident scenarios.
- Visualization libraries like **Matplotlib** and **Seaborn** help display:
    - Severity distribution
    - Impact of weather, road, and vehicle types on severity

5. User Interaction Layer (Telegram Bot)

- A **Telegram bot** is integrated to allow users to input accident details in natural language.
- The bot collects input step-by-step and sends it to the trained model.
- The predicted **severity level** is returned instantly to the user.



Architecture for Accident Detection and Severity Prediction

6. Deployment Layer

- The model and bot are deployed using Python asyncio, Telegram Bot API, and server environments.
- Continuous monitoring and retraining pipelines can be set up for real-time applications.

LIBRARIES:

1. pandas

- **Purpose**: Data loading, cleaning, and manipulation.
- **Usage**:
    - Reading the dataset (CSV format)
    - Handling missing values
    - Selecting relevant features for prediction

2. scikit-learn (sklearn)

A powerful machine learning library used for building, training, and evaluating models.

- **train test split**
    - **Purpose**: Splits data into training and testing sets for validation.
- **One Hot Encoder**
    - **Purpose**: Encodes categorical variables (e.g., weather, road conditions) into numeric form for machine learning.
- **Column Transformer**
    - **Purpose**: Applies different preprocessing steps to numeric and categorical data within a pipeline.
- **Pipeline**
    - **Purpose**: Automates the sequence of preprocessing and model training steps for cleaner and modular code.
- **Random Forest Classifier**
    - **Purpose**: A supervised learning algorithm used for classification using ensemble decision trees.
    - **Role**: Predicts accident severity based on environmental and road conditions.
- **SVC (Support Vector Classifier)**

- **Purpose**: A robust algorithm used for classification with good performance on complex datasets.

◆ **MLP Classifier (Multi-layer Perceptron)**

- **Purpose**: A neural network-based classifier useful for learning non-linear relationships in data.

◆ **classification report, accuracy score:**

- **Purpose**: Evaluates model performance using precision, recall, F1-score, and overall accuracy.

4. telegram exit:

- **Purpose**: Provides extended functionality like command handlers (/start, /run model) and application management.

5. Asyncio:

- **Purpose**: Manages asynchronous operations so the Telegram bot remains responsive while training models.

6. nest Asyncio:

- **Purpose**: Allows asynchronous code (e.g., await, async def) to run within Jupyter notebooks or Google Colab.

SOURCE CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report
import telegram
from telegram import Update
from telegram.ext import ApplicationBuilder, CommandHandler, MessageHandler, filters, ContextTypes
import asyncio
import nest_asyncio # Fix for JupyterNotebook environments
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("/content/RTA Dataset.csv")
# Display the first 5 rows of the dataset
df.head()
# Display the last 5 rows of the dataset
df.tail()
# Shape and column names
print("Dataset shape:", df.shape)
print("Column names:\n", df.columns)
# Data types
print("\nInfo:")
print(df.info())
# Missing values
print("\nMissing values per column:")
print(df.isnull().sum())
```

```
# Summary statistics
print("\nSummary statistics:")
print(df.describe(include='all'))
# See unique values in relevant features
for col in ["Weather_conditions", "Road_surface_conditions", "Type_of_vehicle", "Light_conditions"
,"Accident_severity"]:
    print(f"\nUnique values in '{col}':")
    print(df[col].value_counts())
df = df.dropna(subset=["Accident_severity"])
selected_features = ["Weather_conditions", "Road_surface_conditions", "Type_of_vehicle", "Light_conditions"]
X = df[selected_features]
y = df["Accident_severity"]
numerical_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()
X[numerical_features] = X[numerical_features].fillna(X[numerical_features].mean())
X[categorical_features] = X[categorical_features].fillna("Unknown")
numerical_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown="ignore")

preprocessor = ColumnTransformer([
    ("num", numerical_transformer, numerical_features),
    ("cat", categorical_transformer, categorical_features)
])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Random Forest
print("\nTraining and evaluating: Random Forest")
rf_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(n_estimators=100, random_state=42))
])
```

```
rf_pipeline.fit(X_train, y_train)
rf_pred =rf_pipeline.predict(X_test)

print("Accuracy:", accuracy_score(y_test, rf_pred))
print("Classification Report:\n", classification_report(y_test, rf_pred))
# Support Vector Machine
print("\nTraining and evaluating: Support Vector Machine")
svm_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", SVC(kernel="rbf", C=1.0, gamma="scale"))
])

svm_pipeline.fit(X_train, y_train)
svm_pred = svm_pipeline.predict(X_test)

print("Accuracy:", accuracy_score(y_test, svm_pred))
print("Classification Report:\n", classification_report(y_test, svm_pred))
# Data Visualization
print("\nGenerating visualizations...")

# Set seaborn style
sns.set(style="whitegrid")

# Plot class distribution
plt.figure(figsize=(8, 6))
sns.countplot(data=df, x="Accident_severity", order=df["Accident_severity"].value_counts().index)
plt.title("Accident Severity Distribution")
plt.xlabel("Severity")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

```
# Weather Conditions vs Severity
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x="Weather_conditions", hue="Accident_severity",
order=df["Weather_conditions"].value_counts().index)
plt.title("Accident Severity by Weather Conditions")
plt.xlabel("Weather Conditions")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Road Surface vs Severity
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x="Road_surface_conditions", hue="Accident_severity",
order=df["Road_surface_conditions"].value_counts().index)
plt.title("Accident Severity by Road Surface Conditions")
plt.xlabel("Road Surface Conditions")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Light Conditions vs Severity
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x="Light_conditions", hue="Accident_severity",
order=df["Light_conditions"].value_counts().index)
plt.title("Accident Severity by Light Conditions")
plt.xlabel("Light Conditions")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
```

```
plt.show()


# Vehicle Type vs Severity
plt.figure(figsize=(12, 6))
top_vehicle_types = df["Type_of_vehicle"].value_counts().nlargest(10).index
filtered_df = df[df["Type_of_vehicle"].isin(top_vehicle_types)]
sns.countplot(data=filtered_df, x="Type_of_vehicle", hue="Accident_severity", order=top_vehicle_types)
plt.title("Top 10 Vehicle Types vs Accident Severity")
plt.xlabel("Type of Vehicle")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
BOT_TOKEN = "8046031978:AAFxvBQAmitzpCX7cTCE-SHQuZaGEKxY_Lw"


# There were two definitions of the start and reply functions.
# Keeping the second definition as it includes the bot interaction logic.


user_data = {}


async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_data[update.effective_chat.id] = {"step": 0, "inputs": {}}
    await update.message.reply_text("Welcome! Let's predict accident severity.\nEnter Weather Condition:")


async defreply(update: Update, context: ContextTypes.DEFAULT_TYPE):
    chat_id = update.effective_chat.id
    text = update.message.text


    steps = ["Weather_conditions", "Road_surface_conditions", "Type_of_vehicle", "Light_conditions"]
    prompts = [
        "Enter Road Surface Condition:",
```

```
        "Enter Type of Vehicle:",
        "Enter Light Condition:"
    ] # Corrected indentation here

    if chat_id not in user_data:
        await update.message.reply_text("Please type /start to begin.")
        return

    user_state = user_data[chat_id]

    if user_state["step"] < len(steps):
        current_feature = steps[user_state["step"]]
        user_state["inputs"][current_feature] = text
        user_state["step"] += 1

        if user_state["step"] < len(steps):
            await update.message.reply_text(prompts[user_state["step"] - 1])
        else:
            # All inputs collected, predict
            # You need to select one of the trained models (rf_pipeline, svm_pipeline, nn_pipeline)
            # For example, let's use the Random Forest model:
            model = rf_pipeline # Or svm_pipeline or nn_pipeline
            input_df = pd.DataFrame([user_state["inputs"]])
            prediction = model.predict(input_df)[0]
            await update.message.reply_text(f"Predicted Severity: {prediction}")
            user_data.pop(chat_id)
async def main():
    app =ApplicationBuilder().token(BOT_TOKEN).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(MessageHandler(filters.TEXT, reply))
    await app.run_polling()
```

```
if __name___== '__main__':
    try:
        nest_asyncio.apply() # Fix forenvironments like Jupyter Notebook
        asyncio.run(main())  # Runs the bot correctly
    except RuntimeError:
        loop = asyncio.new_event_loop()
        asyncio.set_event_loop(loop)
        loop.run_until_complete(main())
```

OUTPUT:

```
Dataset shape: (12316, 32)
Column names:
 Index(['Time', 'Day_of_week', 'Age_band_of_driver', 'Sex_of_driver',
       'Educational_level', 'Vehicle_driver_relation', 'Driving_experience',
       'Type_of_vehicle', 'Owner_of_vehicle', 'Service_year_of_vehicle',
       'Defect_of_vehicle', 'Area_accident_occured', 'Lanes_or_Medians',
       'Road_allignment', 'Types_of_Junction', 'Road_surface_type',
       'Road_surface_conditions', 'Light_conditions', 'Weather_conditions',
       'Type_of_collision', 'Number_of_vehicles_involved',
       'Number_of_casualties', 'Vehicle_movement', 'Casualty_class',
       'Sex_of_casualty', 'Age_band_of_casualty', 'Casualty_severity',
       'Work_of_casuality', 'Fitness_of_casuality', 'Pedestrian_movement',
       'Cause_of_accident', 'Accident_severity'],
      dtype='object')

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12316 entries, 0 to 12315
Data columns (total 32 columns):
```

#	Column	Non-Null Count	Dtype
0	Time	12316 non-null	object
1	Day_of_week	12316 non-null	object
2	Age_band_of_driver	12316 non-null	object
3	Sex_of_driver	12316 non-null	object
4	Educational_level	11575 non-null	object
5	Vehicle_driver_relation	11737 non-null	object
6	Driving_experience	11487 non-null	object
7	Type_of_vehicle	11366 non-null	object
8	Owner_of_vehicle	11834 non-null	object
9	Service_year_of_vehicle	8388 non-null	object
10	Defect_of_vehicle	7889 non-null	object
11	Area_accident_occured	12077 non-null	object
12	Lanes_or_Medians	11931 non-null	object
13	Road_allignment	12174 non-null	object
14	Types_of_Junction	11429 non-null	object
15	Road_surface_type	12144 non-null	object
16	Road_surface_conditions	12316 non-null	object
17	Light_conditions	12316 non-null	object
18	Weather_conditions	12316 non-null	object
19	Type_of_collision	12161 non-null	object
20	Number_of_vehicles_involved	12316 non-null	int64
21	Number_of_casualties	12316 non-null	int64
22	Vehicle_movement	12008 non-null	object
23	Casualty_class	12316 non-null	object
24	Sex_of_casualty	12316 non-null	object
25	Age_band_of_casualty	12316 non-null	object
26	Casualty_severity	12316 non-null	object
27	Work_of_casuality	9118 non-null	object
28	Fitness_of_casuality	9681 non-null	object

29 Pedestrian_movement 12316 non-null object

30 Cause_of_accident 12316 non-null object

31 Accident_severity 12316 non-null object

dtypes: int64(2), object(30)

memory usage: 3.0+MB

None

Missing values per column:

Time 0

Day_of_week 0

Age_band_of_driver 0

Sex_of_driver 0

Educational_level 741

Vehicle_driver_relation 579

Driving_experience 829

Type_of_vehicle 950

Owner_of_vehicle 482

Service_year_of_vehicle 3928

Defect_of_vehicle 4427

Area_accident_occured 239

Lanes_or_Medians 385

Road_allignment 142

Types_of_Junction 887

Road_surface_type 172

Road_surface_conditions 0

Light_conditions 0

Weather_conditions 0

Type_of_collision 155

Number_of_vehicles_involved 0

Number_of_casualties 0

Vehicle_movement 308

```
Casualty_class                0
Sex_of_casualty               0
Age_band_of_casualty          0
Casualty_severity             0
Work_of_casuality          3198
Fitness_of_casuality       2635
Pedestrian_movement           0
Cause_of_accident             0
Accident_severity             0
dtype: int64
```

Summary statistics:

```
        Time Day_of_week Age_band_of_driver Sex_of_driver \
count   12316      12316           12316       12316
unique   1074          7               5           3
top   15:30:00     Friday           18-30        Male
freq      120       2041            4271       11437
mean      NaN        NaN             NaN         NaN
std       NaN        NaN             NaN         NaN
min       NaN        NaN             NaN         NaN
25%       NaN        NaN             NaN         NaN
50%       NaN        NaN             NaN         NaN
75%       NaN        NaN             NaN         NaN
max       NaN        NaN             NaN         NaN

       Educational_level Vehicle_driver_relation Driving_experience  \
count              11575                  11737              11487
unique                 7                      4                  7
top     Junior high school               Employee            5-10yr
freq                7619                   9627               3363
mean                 NaN                    NaN                NaN
```

std	NaN	NaN	NaN
min	NaN	NaN	NaN
25%	NaN	NaN	NaN
50%	NaN	NaN	NaN
75%	NaN	NaN	NaN
max	NaN	NaN	NaN

	Type_of_vehicle	Owner_of_vehicle	Service_year_of_vehicle	... \
count	11366	11834	8388	…
unique	17	4	6	...
top	Automobile	Owner	Unknown	…
freq	3205	10459	2883	…
mean	NaN	NaN	NaN	…
std	NaN	NaN	NaN	…
min	NaN	NaN	NaN	…
25%	NaN	NaN	NaN	…
50%	NaN	NaN	NaN	…
75%	NaN	NaN	NaN	…
max	NaN	NaN	NaN	…

	Vehicle_movement	Casualty_class	Sex_of_casualty	Age_band_of_casualty \
count	12008	12316	12316	12316
unique	13	4	3	6
top	Going straight	Driver or rider	Male	na
freq	8158	4944	5253	4443
mean	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN

```
max              NaN           NaN          NaN              NaN


      Casualty_severity Work_of_casuality Fitness_of_casuality  \
count           12316            9118             9681
unique              4               7                5
top                 3          Driver           Normal
freq             7076            5903             9608
mean              NaN             NaN              NaN
std               NaN             NaN              NaN
min               NaN             NaN              NaN
25%               NaN             NaN              NaN
50%               NaN             NaN              NaN
75%               NaN             NaN              NaN
max               NaN             NaN              NaN


      Pedestrian_movement Cause_of_accident Accident_severity
count             12316             12316             12316
unique                9                20                 3
top      Not a Pedestrian     No distancing      Slight Injury
freq              11390              2263             10415
mean                NaN               NaN               NaN
std                 NaN               NaN               NaN
min                 NaN               NaN               NaN
25%                 NaN               NaN               NaN
50%                 NaN               NaN               NaN
75%                 NaN               NaN               NaN
max                 NaN               NaN               NaN


[11 rows x 32 columns]


Unique values in 'Weather_conditions':
```

Weather_conditions

Normal	10063
Raining	1331
Other	296
Unknown	292
Cloudy	125
Windy	98
Snow	61
Raining and Windy	40
Fog or mist	10

Name: count, dtype: int64

Unique values in 'Road_surface_conditions':

Road_surface_conditions

Dry	9340
Wet or damp	2904
Snow	70
Flood over 3cm. deep	2

Name: count, dtype: int64

Unique values in 'Type_of_vehicle':

Type_of_vehicle

Automobile	3205
Lorry (41?100Q)	2186
Other	1208
Pick up upto 10Q	811
Public (12 seats)	711
Stationwagen	687
Lorry (11?40Q)	541
Public (13?45 seats)	532
Public (> 45 seats)	404

```
Long lorry              383
Taxi                    265
Motorcycle              177
Special vehicle          84
Ridden horse             76
Turbo                    46
Bajaj                    29
Bicycle                  21

Name: count, dtype: int64


Unique values in 'Light_conditions':
Light_conditions
Daylight                8798
Darkness - lights lit   3286
Darkness - no lighting   192
Darkness - lights unlit   40
Name: count, dtype: int64


Unique values in 'Accident_severity':
Accident_severity
Slight Injury      10415
Serious Injury      1743
Fatal injury         158
Name: count, dtype: int64


Training and evaluating: Random Forest
<ipython-input-7-7c4aaa38aa70>:45:  SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

X[categorical_features] = X[categorical_features].fillna("Unknown")

Accuracy: 0.836038961038961

Classification Report:

	precision	recall	f1-score	support
Fatal injury	0.00	0.00	0.00	37
Serious Injury	0.30	0.01	0.02	363
Slight Injury	0.84	1.00	0.91	2064
accuracy			0.84	2464
macro avg	0.38	0.33	0.31	2464
weighted avg	0.75	0.84	0.77	2464

Training and evaluating: Support Vector Machine

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

Accuracy: 0.8376623376623377

Classification Report:

	precision	recall	f1-score	support
Fatal injury	0.00	0.00	0.00	37

Serious Injury	0.00	0.00	0.00	363
Slight Injury	0.84	1.00	0.91	2064
accuracy			0.84	2464
macro avg	0.28	0.33	0.30	2464
weighted avg	0.70	0.84	0.76	2464

Generating visualizations...

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.

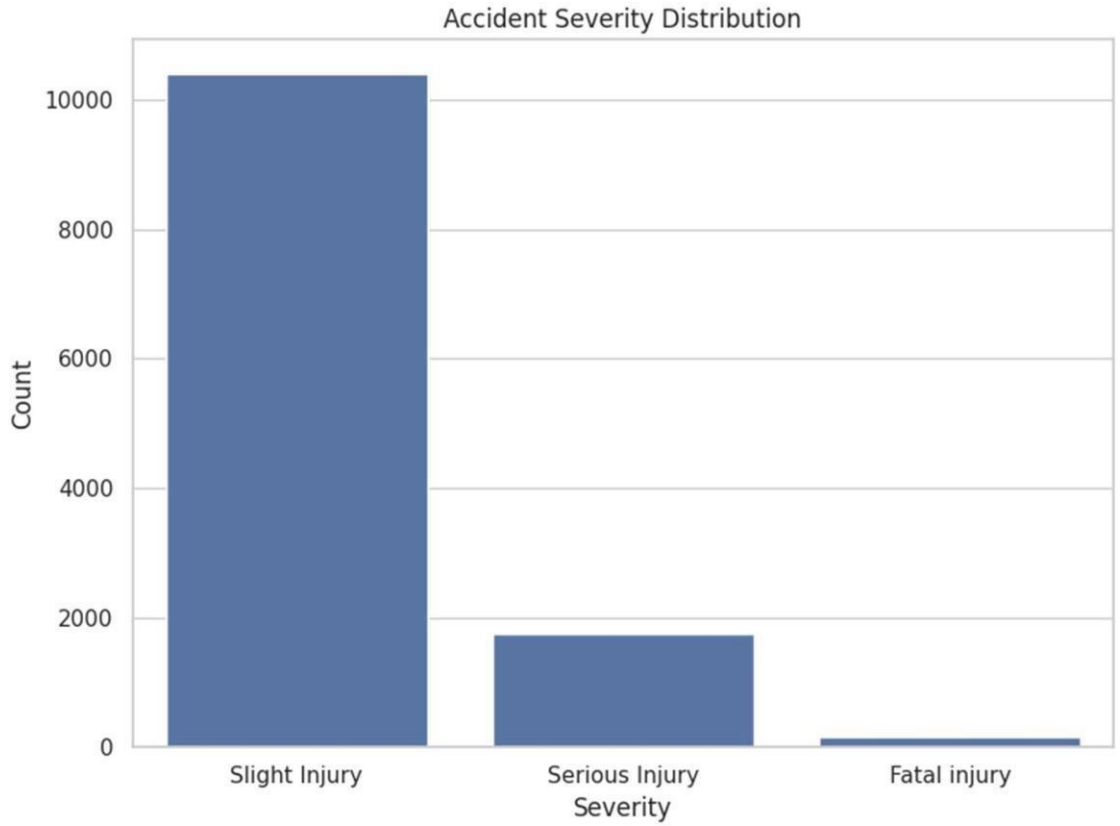
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

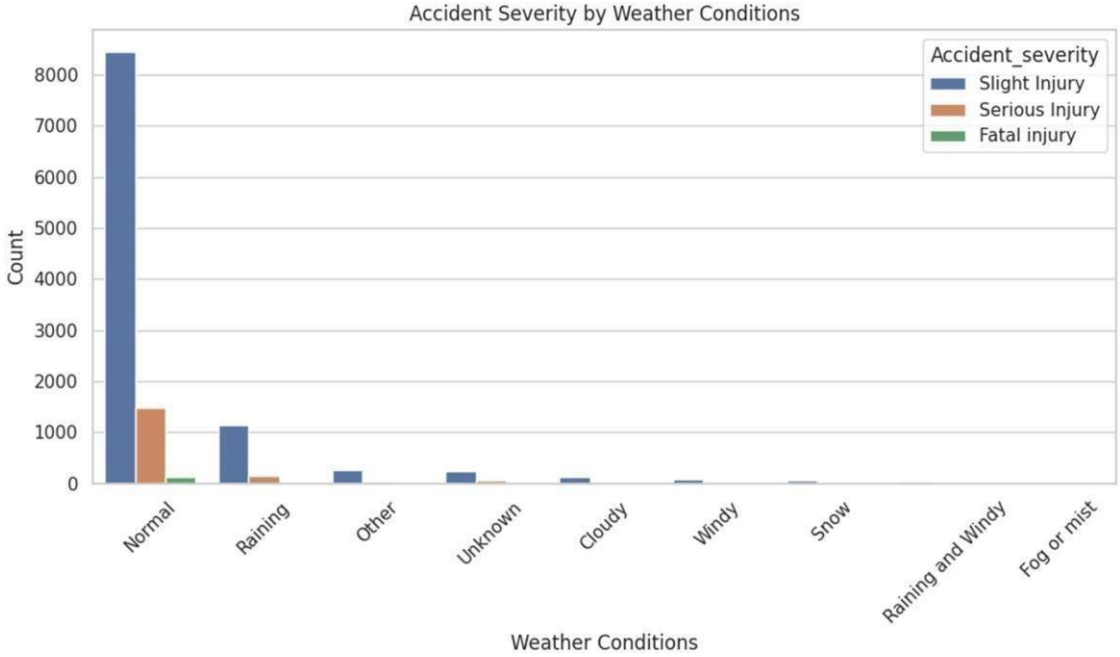
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.

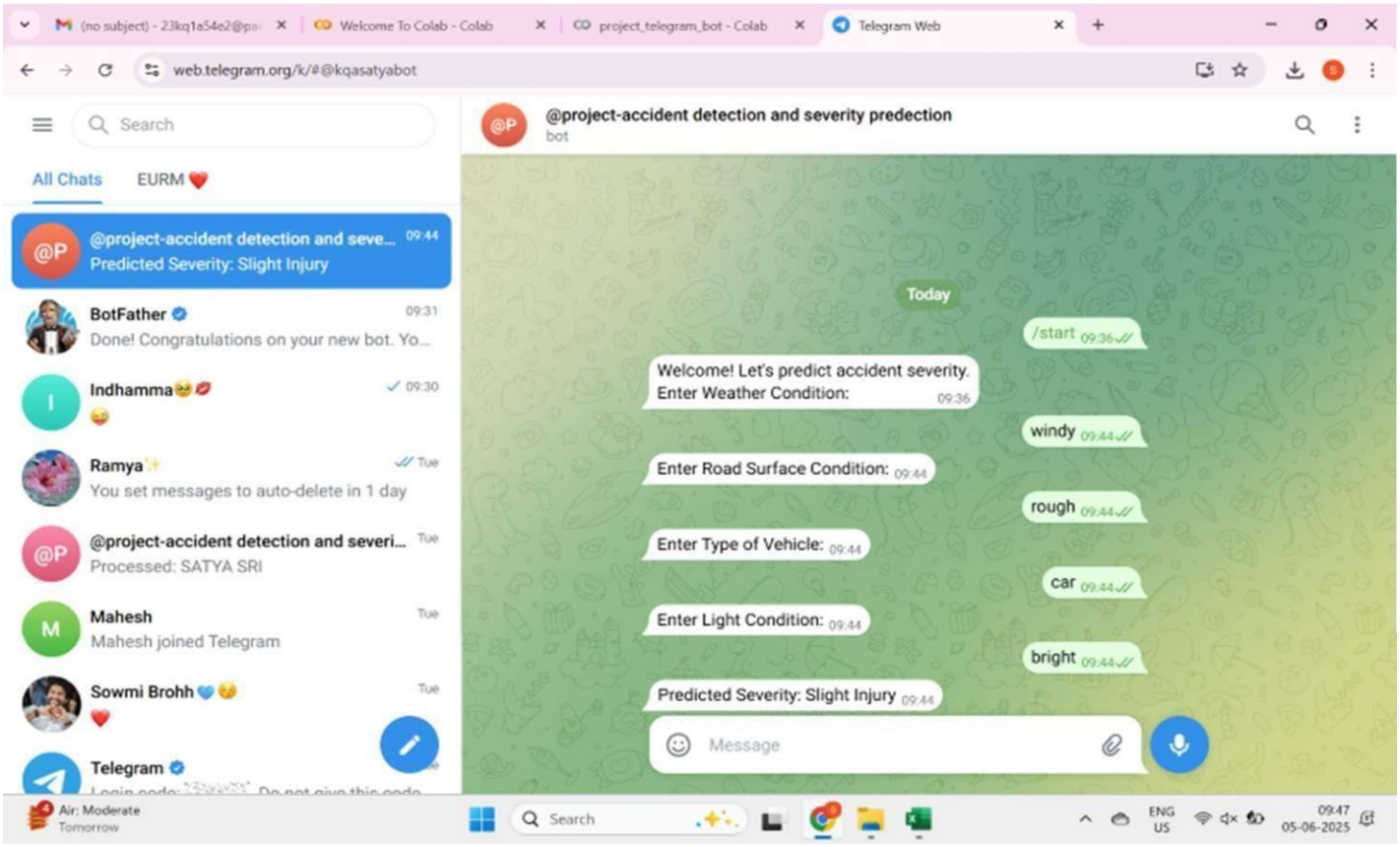
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))


Accident Severity Distribution
Count
10000
8000
6000
4000
2000
0
Slight Injury
Serious Injury
Fatal injury
Severity


Accident Severity by Weather Conditions
Accident_severity
Slight Injury
Serious Injury
Fatal injury
Count
8000
7000
6000
5000
4000
3000
2000
1000
0
Normal
Raining
Other
Unknown
Cloudy
Windy
Snow
Raining and Windy
Fog or mist
Weather Conditions


web.telegram.org/k/#@kqasatyabot
Search
All Chats
EURM
@project-accident detection and seve... 09:44
Predicted Severity: Slight Injury
BotFather 09:31
Done! Congratulations on your new bot. Yo...
Indhamma 09:30
Ramya Tue
You set messages to auto-delete in 1 day
@project-accident detection and severi... Tue
Processed: SATYA SRI
Mahesh Tue
Mahesh joined Telegram
Sowmi Brohh Tue
Telegram
@project-accident detection and severity predection
bot
Today
/start 09:36
Welcome! Let's predict accident severity.
Enter Weather Condition: 09:36
windy 09:44
Enter Road Surface Condition: 09:44
rough 09:44
Enter Type of Vehicle: 09:44
car 09:44
Enter Light Condition: 09:44
bright 09:44
Predicted Severity: Slight Injury 09:44
Message
Air: Moderate
Tomorrow
Search
ENG
US
09:47
05-06-2025

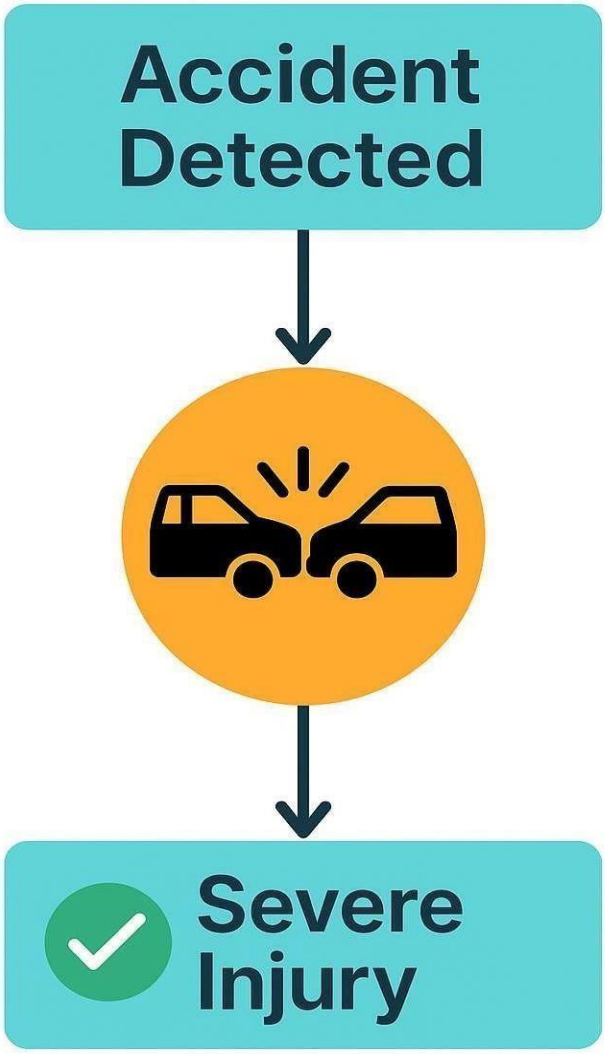
Outcome of the Project:

1. **Successful Prediction of Accident Severity:**
   - Machine Learning models like **Random Forest**, **SVM**, and **Neural Networks** were able to classify accident severity into categories such as *Slight*, *Serious*, and *Fatal* with reasonable accuracy.
2. **Feature Importance Identification:**
   - The models helped identify which features (e.g., **weather conditions, road type, light conditions, vehicle type**) are most influential in predicting the severity of accidents.
3. **Practical Implementation:**
   - A working **ML pipeline** was developed, which can be integrated with traffic monitoring systems or alert services for real-time applications.
4. **Automation Possibility:**
   - The project proved that traffic accident detection and severity prediction can be **automated**, reducing the reliance on manual reporting and improving emergency response times.

What We Learned:

1. **Understanding of Machine Learning Techniques:**
   - Gained hands-on experience with **classification algorithms**, preprocessing techniques, and model evaluation in a real-world context.
2. **Supervised Learning Application:**
   - Understood how **supervised learning** can be used when the output labels (accident severity) are known and how to train models to make predictions on new data.

3. **Data Preprocessing Importance:**
    - Learned the importance of **cleaning, encoding categorical variables**, **scaling numerical features**, and **splitting data** properly for optimal model performance.
4. **Real-World Data Challenges:**
    - Encountered and managed common issues such as **missing values**, **imbalanced classes**, and **noise** in datasets.
5. **Evaluation Metrics Usage:**
    - Learned to use different evaluation metrics like **confusion matrix**, **accuracy**, **precision**, **recall**, and **F1-score** to judge model performance.
6. **Impact of Feature Selection:**
    - Learned how irrelevant or redundant features can negatively affect the model and how proper **feature engineering** boosts accuracy.
7. **Ethical and Social Implications:**
    - Gained awareness about the **ethical responsibility** of using AI in public safety systems and the importance of **data privacy and fairness**.



Final Outcome Diagram